Chapter 4

Accuracy

Once we start using algorithms that return an approximation – be it Taylor's theorem to approximate the value of a function, Simpson's rule to approximate an integrand and hence the integral, or Monte Carlo integration that returns a random number that should be close to the true answer – we encounter error. This short chapter is about how we measure error, and some of the sources of error. We should have some idea of what levels of accuracy are feasible for 'normal' implementations of numbers on computers. And for *iterative* algorithms, that refine the accuracy of an answer step-by-step, we need to measure the rate at which the answer improves.

4.1 Error and Relative Error

Let $u \in \mathbb{R}$ be the number that we want to approximate, and $\tilde{u} \in \mathbb{R}$ its approximation. There are three ways to measure the error:

$$\begin{array}{ll} \tilde{u} - u & \text{is the } \textbf{error},^{1} \\ |\tilde{u} - u| & \text{is the } \textbf{absolute error}, \\ \dfrac{|\tilde{u} - u|}{|u|} & \text{is the } \textbf{relative error}. \end{array}$$

[1] Some books use the opposite sign for the error: $u - \tilde{u}$. But in the spirit of Taylor's theorem we would call this the **remainder** rather than the error.

If we approximate a vector $\boldsymbol{u} \in \mathbb{R}^n$ by $\tilde{\boldsymbol{u}} \in \mathbb{R}^n$, then we would replace the absolute value $|\cdot|$ by the Euclidean norm $\|\cdot\|$. Absolute and relative error are always scalars.

For an example, suppose that we wish to approximate the value of $\sin\left(\frac{355}{113}\right)$ using the Taylor polynomial we derived in **Example 1.6**, but for even more accuracy we decide to use the 14th-order Taylor polynomial

$$\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \frac{x^{11}}{11!} + \frac{x^{13}}{13!}. \tag{4.1}$$

Before starting, we computed an upper bound on the error (the remainder term), which in **Example 1.6** we found was $\frac{x^{15}}{15!}\cos(\xi)$ for some ξ, which is less than $(\frac{355}{113})^{15}/15! \approx 0.000021915381$. The *absolute error* of (4.1) is going to be small.

We feel less good about the error when we compute (4.1) as, to 12 decimal places, 0.000020875830. In fact,

$$\sin\left(\tfrac{355}{113}\right) = -0.000000266764189062419148406 37\cdots$$

and the *relative error* was approximately -79.26! We did not even get the sign correct.

This is a case where the absolute error is small, but the relative error is large. Furthermore, we might try to reduce the relative error by increasing the order of the Taylor polynomial. We can do this to some extent, reducing the relative error to around 10^{-9} with a 26th-order polynomial. But, after that, additional terms do not reduce the (absolute or relative) error at all. This is because of how real numbers are stored with limited precision (see the next section).

Whether relative or absolute error is appropriate depends on the application. We might think that an error of ± 1 is significant if the true value is 2, and insignificant if the true value is $2 \cdot 10^9$: this suggests that relative error is a better quantity, and more advanced books on numerical analysis tend to focus on relative error. On the other hand, in this introductory course we will often only be able to deal with absolute error. It is generally simpler to find bounds on absolute error; you might notice that the error bounds for numerical integration, in Chapter 3, were all in absolute terms.

Something that advanced books also discuss is the distinction between **forward error** and **backward error**. Loosely speaking, when we want to approximate the value of $f(x)$ (for example f integrates something that depends on x, or finds the root of a function

that depends on x), by a numerical algorithm that computes $\tilde{f}(x)$, the forward error is $\tilde{f}(x) - f(x)$ and the backward error is $\tilde{x} - x$ when $\tilde{f}(x) = f(\tilde{x})$. The forward error is how much the approximate answer is wrong, and the backward error is how much you would have to change the input to make the approximate answer right! This does not fall within our syllabus, but the distinction between forward and backward error analysis is important in numerical linear algebra.

4.2 Floating-Point Numbers

Computers rarely store a real number precisely. Although it is conceivable to store arbitrarily many decimal places, it is impossible to store infinitely many. And to keep memory requirements small and predictable, almost every language (and every processor) stores a discrete approximation to a real number called a **floating-point number**.

Just as scientists often write numbers as some multiple of a power of ten, e.g. $c = 2.99792 \cdot 10^8$, the most common computer representations of real numbers are powers of two, of the form

$$\pm\left(1 + \frac{m}{2^p}\right) \cdot 2^e$$

where $0 \leq m < 2^p$ and $e \in \mathbb{Z}$. $\frac{m}{2^p}$ is called the **mantissa**[2] and e is the **exponent**. In the IEEE 754 technical standard for floating-point[3] there is a particular format for storing this representation, which need not concern us. Sufficient to know that m is stored in binary using p bits, e (limited to $e_{min} < e < e_{max}$) using $\lceil \log_2(e_{max} - e_{min}) \rceil$ bits, and the $\pm$ sign using 1 bit. The number of bits in the mantissa, p, is called the **precision**. The standard also specifies representations of error (*Inf* and $-$*Inf* for infinite values, and *NaN* (**not a number**) for the result of calculations like 0/0), and some special cases for numbers very close to zero, which we will not consider.

There are two widely-adopted formats from the IEEE standard:

Single precision takes 4 bytes of storage: the sign bit, 23 bits of precision, and 8 bits for the exponent with $e_{min} = -126$, $e_{max} = 127$. The type associated with single-precision numbers is commonly called `float` (in C-like languages). In Haskell and Scala it is `Float`.

[2]Donald Knuth dislikes this terminology, as it means something else in the context of logarithms. He suggests calling it the 'fraction part'; who am I to disagree?

[3]The latest version is from 2019: `https://ieeexplore.ieee.org/document/8766229`.

Double precision takes 8 bytes of storage: the sign bit, 52 bits of precision, and 11 bits for the exponent with $e_{min} = -1022$, $e_{max} = 1023$. The type associated with double-precision numbers is commonly called `double` (in C-like languages). In Haskell and Scala it is `Double`. Watch out: in Python it is called `float`.

Some processors also use registers that have **extended precision**, taking 10 bytes of memory and supplying 63 bits of precision. There is also a less-widely implemented quadruple-precision format.

It is probably true that the default for computing today is double precision; the exception would be GPUs, where single precision is still common, as 23 bits of mantissa is sufficient resolution for storing colours of textures and pixels.

What does this mean for numerical computing? First, that (unlike integers) there is almost no chance of overflow. The largest positive number (negative numbers are symmetrical) that can be stored in double precision is just under $2 \cdot 2^{1023}$, which is more than 10^{308}. Instead, the danger is of **rounding error**, because only approximately 15 significant decimal digits are stored. Note that the rounding errors of floating-point numbers are relative, not absolute. Understanding when floating-point rounding is going to cause significant errors could be a lecture course in its own right, but here are some condensed pieces of advice.

(i) Because few floating-point numbers are stored exactly, and will be subject to rounding at intermediate stages of computation, it is dangerous to test them for equality. In Scala (on my system, at least), `var x=0.0; while(x!=1.0) x+=0.1;` fails to terminate because `x` just misses the exact value 1.0. Instead of `if(a==b)` it is common to see `if(abs(a-b)<SMALL)`, for some small constant `SMALL`. Choose `SMALL` with some thought.

(ii) The accuracy of floating-point numbers is usually measured by **machine epsilon**[4], the quantity ϵ such that every number $x \in \mathbb{R}$ can be stored as $\tilde{x}$ with relative error no more than ϵ. For single-precision numbers, $\epsilon = 2^{-24} \approx 6 \cdot 10^{-8}$; for double-precision numbers, $\epsilon = 2^{-53} \approx 1.1 \cdot 10^{-16}$. You should never rely on any floating-point computation having a relative error of less than ϵ. Given that most computations involve a sequence of operations, each of which can introduce or compound previous errors, relative errors a few multiples of ϵ are more common. For example, Brent's

[4] Unfortunately there are two competing definitions for machine epsilon, the other twice the value we have defined here. Both are quite widely used. Be careful when reading other sources.

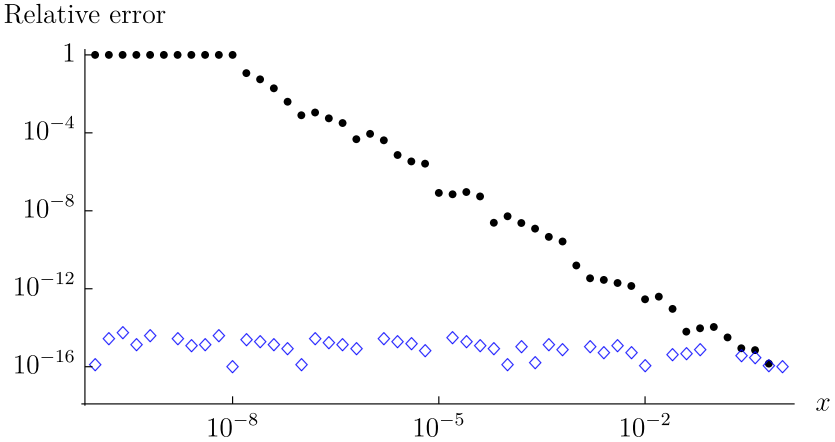

Figure 4.1: Relative error (log-log axes) when computing $1 - \cos x$ (*black dots*) and the equal expression $2\sin^2 \frac{x}{2}$ (*blue diamonds*) in double-precision arithmetic. The latter avoids catastrophic cancellation and has a smaller, more stable, relative error for small x. Note that the more accurate expression still has relative error several multiples of $\epsilon \approx 10^{-16}$.

version of the famous `zeroin` one-dimensional root-finder guarantees relative error of no more than 4ϵ.

(iii) Particular peril lurks where the result of a computation is orders of magnitude smaller than the inputs, because then small relative errors on the inputs can lead to large relative errors on the outputs. A notoriously nasty operation is subtraction: if $a \approx b$ then $a - b \approx 0$, so small *relative* errors in a and/or b can cause large *relative* errors in $a - b$. When this happens it is called **catastrophic cancellation**. Sometimes catastrophic cancellation can be avoided simply be replacing one formula by another, equivalent, formula that avoids subtracting close numbers. For example, in Figure 4.1 we compute $1 - \cos x$ and the equivalent formula $2\sin^2 \frac{x}{2}$, which avoids subtracting close numbers. For small x, the latter is much more accurate. On the other hand, if this occurs in a context like $\sqrt{2} + 1 - \cos x$ then there is probably no need to worry about the cancellation, as the error will remain small relative to the overall answer.

4.3 Sources of Error

What are the sources of error in numerical computing? There are many, and different authors categorize them in different ways. We will mention two main types of error:

Truncation error (in some contexts called discretization error or procedural error) is the error introduced when approximating an infinite procedure by a finite procedure. For example, when we use a finite Taylor polynomial to approximate a function, we have *truncated* a power series and the remainder term is the truncation error. Or when we perform numerical integration we have truncated a hypothetical infinite sum (perfectly accurate for continuous functions) by the finite sum over n strips.

Roundoff error (also called numerical error) is error introduced by approximating a real number, for example in floating-point format. A common mistake is to think that roundoff error is called truncation error because it involves truncating the binary representation of a floating-point number. Roundoff error is **not** truncation error.

We could also identify other sources of error: model error, if we are computing the output of a model but the model itself is an approximation (e.g. in fluid dynamics); input error (if the data fed to our algorithm is measured with some error); programmer error. Some texts also mention **propagation error**, which refers to errors arising in an early part of a numerical method being amplified by a later part. This phenomenon is called **instability** and some classic problems are subject to it. For some $\mathbf{A}$, solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ can be very unstable.

To illustrate a problem with two sources of error, let us approximate $\frac{\mathrm{d}f}{\mathrm{d}x}$ by the **numerical derivative**

$$\tilde{d}(x,h) = \frac{f(x+h) - f(x)}{h}.$$

By definition, $\tilde{d}(x,h)$ tends to $\frac{\mathrm{d}f}{\mathrm{d}x}$ as $h \to 0$. So we approximate $\frac{\mathrm{d}f}{\mathrm{d}x}$ by $\tilde{d}(x,h)$ with small, nonzero, h. (Numerical differentiation is a topic in its own right, but cannot fit into this course.) Let us apply this method to the function $f(x) = e^x$, at $x = 0$. Of course, we know the true answer $\frac{\mathrm{d}f}{\mathrm{d}x}(0) = 1$, so the relative error equals the absolute error in this instance. Implementing $\frac{f(x+h)-f(x)}{h}$ on a computer, we show this error for values of h between 10^{-16} and 1. Reading the graph from right-to-left, observe that the error reduces as h reduces, until h gets to a certain level; then it *increases*. The error for large

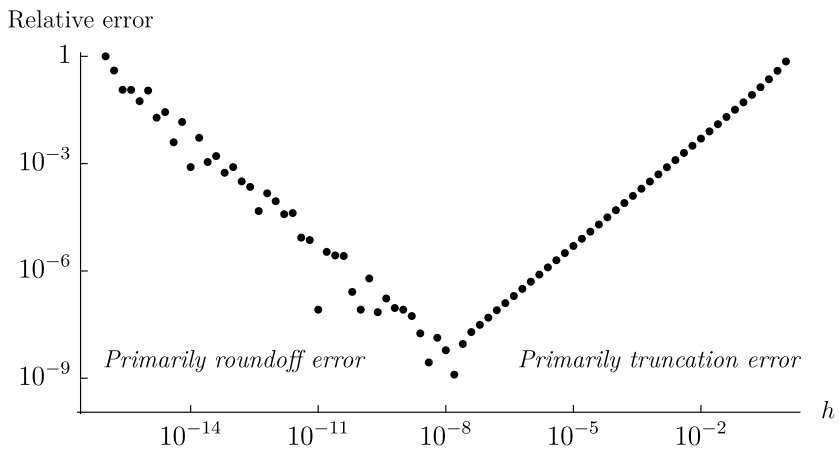

Figure 4.2: Relative error (log-log axes) when estimating $\frac{\mathrm{d}f}{\mathrm{d}x}$ by $\frac{f(x+h)-f(x)}{h}$, for $f(x) = e^x$ and $x = 0$. Note that relative error equals absolute error for this problem. For h not too close to zero, most of the error is truncation error (approximating a limit by a small finite value). When h gets small enough, the relative error is mainly caused by roundoff in $f(x+h) - f(x)$; for very small h the cancellation is catastrophic.

h is mainly due to truncation error, approximating a limit by a small finite value. The error for small h is due to roundoff in calculating $f(x+h) - f(x)$, which is an example of catastrophic cancellation: a small relative error in $f(x+h)$ leads to a large relative error in $f(x+h) - f(x)$ because $f(x+h) - f(x)$ is close to zero. For example, although $f(10^{-16})$ is actually just over $1 + 10^{-16}$, its double-precision floating point representation is 1, so $f(10^{-16}) - f(0)$ comes out to zero.

Truncation error is usually something that we can analyse, and pick suitable parameters to keep it small. In the above example, we could use the remainder of Taylor's theorem to show that $h < 10^{-6}$ should give relative truncation error no more than $5 \cdot 10^{-7}$, for example. But, as we said earlier, roundoff is more difficult to analyze. Scientists using numerical computing have to spot the types of computation where roundoff error is significant, such as catastrophic cancellation.

4.4 Rates of Convergence

Suppose that we have designed an **iterative numerical algorithm**: a procedure that attempts to approximate a true value x^* (or vector $\boldsymbol{x}^*$) by starting with an initial guess

then repeatedly refining the answer, (hopefully) making it more and more accurate. We will see many such algorithms in Chapters 5 and 6. Starting with some x_0 or $\boldsymbol{x_0}$[5], let

$$x_{n+1} = f(x_n), \quad \text{or} \quad \boldsymbol{x_{n+1}} = \boldsymbol{f}(\boldsymbol{x_n}).$$

We identify the absolute error at the n-th step as

$$\epsilon_n = x_n - x^*, \quad \text{or} \quad \epsilon_n = \|\boldsymbol{x_n} - \boldsymbol{x^*}\|.$$

The first goal of our algorithm is that it should converge to the correct answer: $|\epsilon_n| \to 0$ as $n \to \infty$. But how to compare two methods, both of which achieve this goal? In numerical algorithms, just as in exact algorithms, we want to look at the time complexity: in our case, the time complexity of computing x_{n+1} from x_n. But we also need to look at how *quickly* does $|\epsilon_n| \to 0$. The speed of convergence is measured in the following way.

Definition Let ϵ_n be the error of some iterative numerical algorithm A_n after n steps. If $|\epsilon_n| \to 0$ we say that

(i) A_n **converges linearly** if, for some $0 < a < 1$, $\dfrac{|\epsilon_{n+1}|}{|\epsilon_n|} \to a$.

(ii) A_n **converges sublinearly** if $\dfrac{|\epsilon_{n+1}|}{|\epsilon_n|} \to 1$,

a special case, **logarithmic convergence**, if also $\dfrac{|\epsilon_{n+2} - \epsilon_{n+1}|}{|\epsilon_{n+1} - \epsilon_n|} \to 1$.

(iii) A_n **converges superlinearly** if $\dfrac{|\epsilon_{n+1}|}{|\epsilon_n|} \to 0$,

a special case, **order-q convergence**, if $\dfrac{|\epsilon_{n+1}|}{|\epsilon_n|^q} \to a$ for $a > 0, q > 1$.

(All limits as $n \to \infty$. Note that q need not be an integer.)

Order-2 convergence is also known as **quadratic convergence**. Linear convergence is sometimes known as order-1 convergence.

[5] Understanding the region of starting points, for which the iteration will converge to the right answer, is a key part of numerical computing.

We might measure the performance of a numerical algorithm in terms of its actual error, or alternatively in terms of an upper bound on its error (as we did in Chapter 3). The latter is more appropriate when the algorithm approaches the true answer non-monotonically, in which case the error might go up before it goes down again, all while still tending to zero. In some literature the convergence of the actual error is called Q-linear convergence, Q-quadratic convergence, etc., while the convergence of an error *bound* is called R-linear convergence, R-quadratic convergence, etc. We will not need such a distinction.

We can also use slightly relaxed definitions that do not require the ratio of successive errors to converge to a limit, merely to have an upper bound. For example, if

$$\frac{|\epsilon_{n+1}|}{|\epsilon_n|} < a, \text{ for } a < 1,$$

then we have **at least linear convergence**, and if

$$\frac{|\epsilon_{n+1}|}{|\epsilon_n|^q} < a, \text{ for } a > 0, q > 1,$$

then we have **at least order-q convergence**. They are 'at least because the limit of the ratio might be zero.

Example 4.1 For $c < 1$,

(i) $x_n = \frac{1}{n}$ converges **logarithmically** to zero.
(ii) $x_n = c^n$ converges **linearly** to zero.
(iii) $x_n = c^{n^2}$ converges **superlinearly**, but not with order q for any $q > 1$, to zero.
(iv) $x_n = c^{2^n}$ converges **quadratically** to zero.

The words *quadratic* and *linear* do not mean that the error itself is a quadratic or linear function of n. Instead, it is best to understand a linearly convergent algorithm as one that increases the number of correct decimal places ($-\log_{10}|\epsilon_n|$) by a constant at each step, and a quadratically convergent algorithm as one that approximately *doubles* the number of correct decimal places at each step, provided that the error is fairly small to begin with. Quadratic algorithms are highly prized because they need so few steps to

achieve accuracy that reaches the limits of rounding. However, it is a common property of quadratically convergent numerical algorithms that they *fail* to converge unless the starting point is chosen carefully.

When comparing a slower algorithm with a fast convergence against a fast algorithm with slow convergence, you should bear in mind:

Lemma 4.2 If A_n converges with order q, then A_{2n} converges with order q^2.

Proof

$$\frac{|\epsilon_{2(n+1)}|}{|\epsilon_{2n}|^{q^2}} = \frac{|\epsilon_{2n+2}|}{|\epsilon_{2n+1}|^q} \cdot \left(\frac{|\epsilon_{2n+1}|}{|\epsilon_{2n}|^q}\right)^q \to a^{q+1} \quad \text{if} \quad \frac{|\epsilon_{n+1}|}{|\epsilon_n|^q} \to a.$$

■

So, multiplicative constants aside, given an order-1.5 numerical procedure where each step runs twice as fast as a quadratic procedure, the former is probably preferable. It can achieve order-2.25 in equivalent time simply by taking twice as many steps. Of course, our decision could be different if we take into account to numerical stability of the methods, or whether one method is guaranteed to converge starting from a wider range of starting points.

Example 4.3 The iteration

$$x_{n+1} = \frac{x_n}{2} + \frac{1}{x_n},$$

starting with $x_0 = 1$, converges quadratically to $\sqrt{2}$.

(In fact this is true for any $x_0 > 0$, and for any $x_0 < 0$ it converges quadratically to $-\sqrt{2}$.)

4.5 Bonus Mathematics: Acceleration Methods

This is <u>not</u> part of the syllabus.

If a scalar sequence $(x_0, x_1, \ldots)$ converges linearly, we might expect that

$$\frac{x_{n+2} - x^*}{x_{n+1} - x*} \approx \frac{x_{n+1} - x^*}{x_n - x*} \approx a.$$

Setting it to equality and rearranging gives a 'solution' for x^* which is typically more accurate than x_{n+2}, and can be used instead. This is one of many **acceleration methods**, which by clever manipulation can turn a convergent iteration into one that converges more rapidly. In other contexts (notably integration) they are also called **extrapolation methods**, because they can be seen as extrapolating a sequence of approximations to the point at which it appears to be converging.

This is probably the simplest acceleration method, known as **Aitken extrapolation** or the Δ^2 **method**:

Definition If $x_0, x_1, \ldots$ is a sequence in $\mathbb{R}$ then the sequence

$$\tilde{x}_n = x_n - \frac{(x_{n+1} - x_n)^2}{(x_{n+2} - x_{n+1}) - (x_{n+1} - x_n)}$$

(where $\tilde{x}_n$ can be computed once x_{n+2} has been) is called the Δ^2 **process**.

Note that this generates a parallel sequence $(\tilde{x}_n)$, from (x_n).

Extrapolation methods, like all numerical methods, come with conditions and convergence results. For example,

---

Theorem 4.4 If $x_0, x_1, \ldots$ converges linearly to x^* then the Δ^2 process converges strictly faster, in the sense that

$$\lim_{n\to\infty} \frac{\tilde{x}_n - x^*}{x_n - x^*} = 0.$$

Proof Set

$$a_n = \frac{x_{n+1} - x^*}{x_n - x^*};$$

the hypothesis of linear convergence is that $a_n \to a$ for some $0 < a < 1$. Then calculate

$$\begin{aligned}
\frac{\tilde{x}_{n+1} - x^*}{x_n - x^*} &= \frac{x_n - \frac{(x_{n+1}-x_n)^2}{(x_{n+2}-x_{n+1})-(x_{n+1}-x_n)} - x^*}{x_n - x^*} \\
&= 1 - \frac{(x_{n+1} - x_n)^2}{(x_n - x^*)(x_{n+2} - 2x_{n+1} + x_n)} \\
&= 1 - \frac{\left(\frac{(x_{n+1}-x^*)-(x_n-x^*)}{x_n-x^*}\right)^2}{\frac{(x_{n+2}-x^*)-2(x_{n+1}-x^*)+(x_n-x^*)}{x_n-x^*}} \\
&= 1 - \frac{(a_n - 1)^2}{a_{n+1}a_n - 2a_n + 1} \\
&\to 1 - \frac{(a-1)^2}{a^2 - 2a + 1} = 0.
\end{aligned}$$

■

In some cases, Aitken extrapolation can generate a quadratically convergent sequence. An example is Steffensen's method, which is an iterative algorithm for approximating the root of a univariate function: the subject of chapter 5.

The formula for Aitken extrapolation must be treated with care. It is less numerically stable if the denominator is simplified to

$$(x_{n+2} - x_{n+1}) - (x_{n+1} - x_n) = x_{n+2} - 2x_{n+1} + x_n,$$

and suffers from catastrophic cancellation if the entire formula is 'simplified' to

$$\tilde{x}_n = \frac{x_n x_{n+2} - x_{n+1}^2}{x_{n+2} - 2x_{n+1} + x_n}.$$

Practice Questions

4.1 Using the results in Section 3.4, find upper bounds for the absolute and relative errors when integrating

$$\int_0^1 x^4 + \tfrac{11}{47}x^3 - \tfrac{38}{49}x^2 \,\mathrm{d}x$$

using Simpson's rule with n strips.

4.2 In the **quadratic formula** for solving $ax^2 + bx + c = 0$,

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}, \qquad (*)$$

identify possible catastrophic cancellation when (i) $b^2 - 4ac$ is many orders of magnitude smaller than b^2, and (ii) b^2 is many orders of magnitude greater than $4ac$.

Given the choice between $(*)$ and the equivalent so-called **citardauq formula**,

$$x = \frac{2c}{-b \mp \sqrt{b^2 - 4ac}}, \qquad (\dagger)$$

which would you use?

4.3 Determine the rate of convergence of a hypothetical iterative numerical algorithm where the error after n steps, ϵ_n, is:

(i) $3/(n+3)$, (ii) $3^{-n}n^{-3}$, (iii) $3^{(-3^n - n^3)}$, (iv) $1/(3n)!$.

4.4 Determine the rate of convergence of a hypothetical iterative numerical algorithm where the error after n steps ϵ_n, satisfies the recurrences:

(i) $|\epsilon_{n+1}| = 3|\epsilon_n^3|,\ |\epsilon_0| < \frac{1}{\sqrt{3}}$ (ii) $|\epsilon_{n+1}| = \frac{n+1}{n+2}|\epsilon_n|,\ |\epsilon_0| < 1$ (iii) $\frac{|\epsilon_{n+2}|}{|\epsilon_{n+1}(\epsilon_n)^6|} \to 1$.

Hint: for (iii), assume $\frac{|\epsilon_{n+2}|}{|\epsilon_{n+1}|^q} \to a$, *and solve for* q *and* a.

4.5 The iteration $x_0 = 1$,

$$x_{n+1} = \frac{x_n(x_n^2 + 6)}{3x_n^2 + 2}$$

converges to $\sqrt{2}$ faster than **Example 4.3**; determine its order of convergence. Which of the two methods would you prefer to approximate $\sqrt{2}$ on a computer without a square-root function?

Answers to Chapter 4 Practice Questions

4.1 The exact answer is $\int_0^1 x^4 + \frac{11}{47}x^3 - \frac{38}{49}x^2 \,\mathrm{d}x = \frac{1}{5} + \frac{11}{47\cdot 4} - \frac{38}{49\cdot 3} = \frac{1}{138180} \approx 7.237 \cdot 10^{-6}$. The fourth derivative of the integrand is (the constant) 24. By (3.13), the absolute error is bounded by $\frac{(1-0)^5}{180n^4}24 = \frac{2}{15n^4}$. The relative error is the absolute error divided by $\frac{1}{138180}$, which simplifies to $\frac{18424}{n^4}$.

4.2 (i) If $b^2 - 4ac \approx 0$ then $b^2 - 4ac$ looks like potential catastrophic cancellation. In fact it should not be, because the answer is next added to $-b$, which is orders of magnitude larger, so a small absolute (even if large relative) error in $b^2 - 4ac$ will not cause a large relative error in the overall answer.
(ii) But if $b^2 \gg 4ac$, for the lower root the subtraction $-b + \sqrt{b^2 - 4ac}$ could be catastrophic. It could lead to a large relative error in the answer. In that case the equivalent formula avoids the close-to-zero subtraction.
The citardauq formula suffers from the same problem at the upper root. Assuming $b > 0$ (for $b < 0$ the answers are reversed), the two roots are most stably given by

$$\frac{-b - \sqrt{b^2 - 4ac}}{2a} \quad \text{and} \quad \frac{2c}{-b - \sqrt{b^2 - 4ac}}.$$

4.3 (i) Logarithmic. (ii) Linear. (iii) Order-q for any $q < 3$, but not order-3. (iv) Superlinear, but not order-q for any $q > 1$.

4.4 (i) Cubic. (ii) Logarithmic. (iii) Dividing $\frac{|\epsilon_{n+2}|}{|\epsilon_{n+1}(\epsilon_n)^6|} \to 1$ by $\frac{|\epsilon_{n+2}|}{|\epsilon_{n+1}|^q} \to a$ gives $\frac{|\epsilon_{n+1}|^{q-1}}{|\epsilon_n|^6} \to \frac{1}{a}$, hence $\frac{|\epsilon_{n+1}|}{|\epsilon_n|^{6/(q-1)}} \to \frac{1}{a^{1/(q-1)}}$. This means $q = 6/(q-1)$, hence $q = 3$: cubic convergence.

4.5

$$\frac{x_{n+1} - \sqrt{2}}{(x_n - \sqrt{2})^q} = \frac{\frac{x_n(x_n^2+6)}{3x_n^2+2} - \sqrt{2}}{(x_n - \sqrt{2})^q} = \frac{x_n^3 - 3\sqrt{2}x_n^2 + 6x_n - 2\sqrt{2}}{(3x_n^2+2)(x_n - \sqrt{2})^q} = \frac{(x_n - \sqrt{2})^3}{(3x_n^2+2)(x_n - \sqrt{2})^q}$$

so with $q = 3$

$$\frac{x_{n+1} - \sqrt{2}}{(x_n - \sqrt{2})^q} = \frac{1}{3x_n^2 + 2}$$

which tends to $\frac{1}{5}$, given that $x_n \to \sqrt{2}$. The convergence is cubic.
But two steps of **Example 4.3** would have order-4 convergence, and should be faster (two reciprocals are required for two steps of **Example 4.3**, but fewer multiplications).